A. MỤC TIÊU

- Giới thiệu về SQLite
- Hướng dẫn cách **tích hợp SQLite vào Flutter** bằng thư viện sqflite.
- Hướng dẫn **thao tác với SQLite**: tạo bảng, thêm, sửa, xoá, truy vấn dữ liệu.

DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

B. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Giới thiệu về SQLite

SQLite là một hệ quản trị cơ sở dữ liệu **nhẹ, nhanh, và không cần máy chủ,** có sẵn trên cả Android và iOS. Nó sử dụng các bảng gồm hàng và cột và yêu cầu kiến thức về ngôn ngữ SQL để thực hiện các truy vấn như SELECT, INSERT, UPDATE, DELETE. Nó lưu trữ dữ liệu dưới dạng **file cục bộ**, rất phù hợp với ứng dụng di động.

Đặc điểm của SQLite:

- Không cần cài đặt server, mọi thứ chạy trên file .sqlite
- Nhẹ, hiệu năng cao
- Hỗ trợ truy vấn SQL đầy đủ (SELECT, INSERT, UPDATE, DELETE, JOIN...)

Comment trong SQLite bắt đầu bằng hai ký tự "-" liên tiếp hoặc “/*” và mở rộng đến tiếp tục và bao gồm cặp ký tự “*/”.

Tất cả các câu lệnh trong SQLite bắt đầu với bất kỳ từ khóa nào như SELECT, INSERT, UPDATE, DELETE, ALTER, DROP, v.v. và tất cả các câu lệnh kết thúc bằng dấu chấm phẩy (;).

Để làm việc với SQLite trong Flutter cần phải khai báo các dependencies:

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.4.2
  path_provider: ^2.1.5
  intl: ^0.20.2
```

1.2. Làm việc với SQLite trong Flutter

❖ **Tạo database**

```
openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        // Create the tasks table
        await db.execute('''
          CREATE TABLE tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            description TEXT,
            isCompleted INTEGER
          )
        ''');
      },
    );
```

❖ **Truy vấn dữ liệu trong sqlite có thể thực hiện bằng 2 cách:**

- Truy vấn dữ liệu thông qua phương thức **rawQuery**(): sử dụng các phát biểu SQL; cho phép linh hoạt trong truy vấn.

Cú pháp:

```
db.rawQuery(
  'SELECT * FROM table WHERE id = ?',
  [1],
);
```

- Truy vấn dữ liệu thông qua phương thức **query**(): sử dụng API có sẵn, dễ dàng thao tác với dữ liệu mà không cần viết câu SQL tuy nhiên không cho phép JOIN nhiều bảng linh hoạt.

Cú pháp:

```
db.query(
  'table',
  columns: [...],
  where: 'id = ?',
  whereArgs: [1],
  orderBy: 'name ASC',
);
```

Ý nghĩa các tham số:

Tham số	Ý nghĩa
table	Tên bảng
columns	Các cột cần lấy
where	Điều kiện
whereArgs	Giá trị thay thế ?
orderBy	Sắp xếp

Ví dụ:

```
final result = await db.query(
'hoatdong',
columns: ['id', 'tenhd', 'diemcong'],
where: 'diemcong > ?',
whereArgs: [5],
orderBy: 'ngaybd DESC',
);
```

1.3. Các bước xây dựng ứng dụng cơ sở dữ liệu cục bộ SQLite trong Flutter theo mô hình MVC

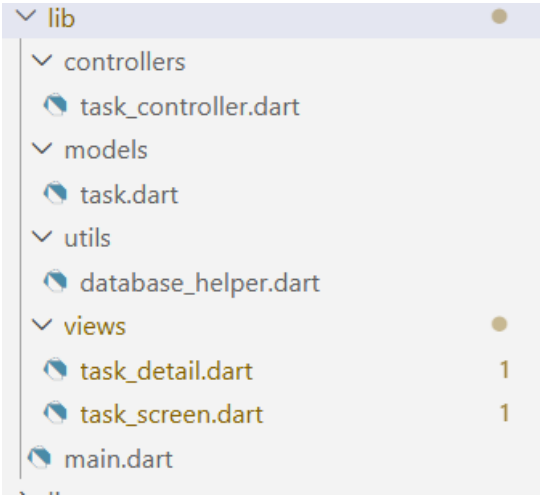
- Bước 1: Tạo model đại diện cho các bảng trong CSDL
- Bước 2: Tạo lớp Datahelper để kết nối CSDL
- Bước 3: Xây dựng các controller thực hiện các thao tác thêm, xóa, sửa
- Bước 4: Xây dựng UI

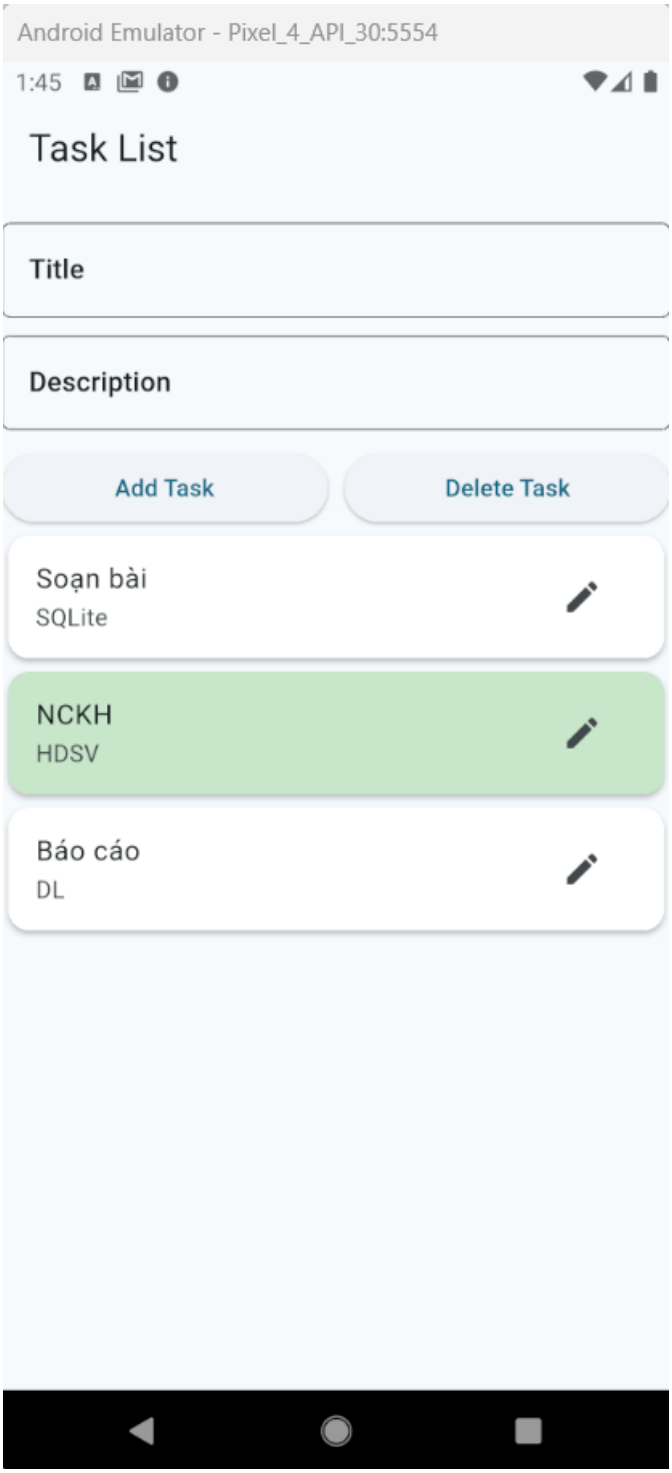
2. Bài tập hướng dẫn

Một công việc có các thông tin: Id, title, description, isComplete (status). Hãy tạo ứng dụng quản lý công việc có giao diện như hình bên.

Hướng dẫn:

Tạo cấu trúc project như sau:





❖ Bước 1: Tạo model cho công việc: **lib/models/task.dart** có nội dung như sau:

```
//create class Task with properties id, title, description, isCompleted
class Task {
  int id;
  String title;
  String description;
  bool isCompleted;
  Task({
    required this.id,
```

```
    required this.title,
    required this.description,
    this.isCompleted = false,
  });
  //convert Task to Map
  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'title': title,
      'description': description,
      'isCompleted': isCompleted ? 1 : 0,
    };
  }
  //convert Map to Task
  factory Task.fromMap(Map<String, dynamic> map) {
    return Task(
      id: map['id'],
      title: map['title'],
      description: map['description'],
      isCompleted: map['isCompleted'] == 1,
    );
  }
}
```

❖ Bước 2: Tạo **lib/utils/database_helper.dart** để kết nối dữ liệu có nội dung như sau:

```
import 'package:sqflite/sqflite.dart';

class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._int();
  static Database ? _database;
  DatabaseHelper._int();
  factory DatabaseHelper() => _instance;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database?> _initDatabase() async {
    // Get the path to the database file
    String path = await getDatabasesPath() + 'tasks.db';
    // Open the database, creating it if it doesn't exist
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        // Create the tasks table
```

```
        await db.execute('''
          CREATE TABLE tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            description TEXT,
            isCompleted INTEGER
          )
        ''');
      },
    );
  }
}
```

❖ Bước 3: tạo **lib/controllers/task_controller.dart** gồm các phương thức CRUD

```
class TaskController {
  final DatabaseHelper _dbHelper = DatabaseHelper();
  // Insert a new task into the database
  Future<void> insertTask(Task task) async {
    final db = await _dbHelper.database;
    await db.insert('tasks', task.toMap());
  }

  // Update an existing task in the database
  Future<void> updateTask(Task task) async {
    final db = await _dbHelper.database;
    await db.update('tasks', task.toMap(), where: 'id = ?', whereArgs:
[task.id]);
  }

  // Delete a task from the database
  Future<void> deleteTask(int id) async {
    final db = await _dbHelper.database;
    await db.delete('tasks', where: 'id = ?', whereArgs: [id]);
  }
  //check task is exist in the database by title
  Future<bool> isTaskExist(String title) async {
    final db = await _dbHelper.database;
    final List<Map<String, dynamic>> maps = await db.query('tasks', where:
'title = ?', whereArgs: [title]);
    return maps.isNotEmpty;
  }
//search task by title or description
  Future<List<Task>> searchTasks(String query) async {
    final db = await _dbHelper.database;
    final List<Map<String, dynamic>> maps = await db.query(
      'tasks',
      where: 'title LIKE ? OR description LIKE ?',
```

```
      whereArgs: ['%$query%', '%$query%'],
    );
    return maps.map((obj) => Task.fromMap(obj)).toList();
  }
  // Fetch all tasks from the database
  Future<List<Task>> fetchTasks() async {
    final db = await _dbHelper.database;
    final List<Map<String, dynamic>> maps = await db.query('tasks');
    return maps.map((obj) => Task.fromMap(obj)).toList();
  }
}
```

❖ Bước 4: tạo UI **lib/views/task_screen.dart** như hình trên và gọi các phương thức xử lý tương ứng

```
//create task screen with list of tasks and add task button and edit task
button
import 'package:flutter/material.dart';
import 'package:lab9_flutter_sqlite/controllers/task_controller.dart';
import 'package:lab9_flutter_sqlite/models/task.dart';
import 'package:lab9_flutter_sqlite/views/task_detail.dart';

class TaskScreen extends StatefulWidget {
  @override
  _TaskScreenState createState() => _TaskScreenState();
}

class _TaskScreenState extends State<TaskScreen> {
  List<Task> _tasks = [];
  final TaskController _taskController = TaskController();
  final titleController = TextEditingController();
  final descriptionController = TextEditingController();
  final TextEditingController _searchQuery = TextEditingController();
  @override
  void initState() {
    super.initState();
    _fetchTasks();
  }

  Future<void> _fetchTasks() async {
    final tasks = await _taskController.fetchTasks();
    setState(() {
      _tasks = tasks;
    });
  }

  @override
  Widget build(BuildContext context) {
```

```
    TextStyle textStyle = Theme.of(context).textTheme.titleMedium!;
    return Scaffold(
      appBar: AppBar(title: Text('Task List')),
      body: // create two textfields for title and description and a
button to add task and list of tasks with edit button
      Padding(
        padding: EdgeInsets.only(top: 15.0, bottom: 15.0),
        child: Column(
          children: [
            TextField(
              controller: titleController,
              decoration: InputDecoration(
                labelText: 'Title',
                labelStyle: textStyle,
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(5.0),
                ),
              ),
              onTap: () => {},
            ),
            SizedBox(height: 10.0),
            TextField(
              controller: descriptionController,
              decoration: InputDecoration(
                labelText: 'Description',
                labelStyle: textStyle,
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(5.0),
                ),
              ),

              onTap: () => {},
            ),
            SizedBox(height: 10.0),
            Row(
              mainAxisAlignment: MainAxisAlignment.end,
              children: [
                Expanded(
                  child: ElevatedButton(
                    onPressed: () {
                      // Add task logic
                      //save task to database and refresh task list
                      Task newTask = Task(
                        id: _tasks.isNotEmpty ? _tasks.last.id + 1 : 1,
                        title: titleController.text,
                        description: descriptionController.text,
                      );
                      _taskController.insertTask(newTask).then((_) {
                        _fetchTasks();
```

```
                        descriptionController.clear();
                        titleController.clear();
                      });
                    },
                    child: Text('Add Task'),
                  ),
                ),
                SizedBox(width: 10.0),
                Expanded(
                  child: ElevatedButton(
                    onPressed: () {
                      // Delete task logic
                      //delete task from database and refresh task list
                      if (_tasks.isNotEmpty) {
                        Task lastTask = _tasks.last;
                        _taskController.deleteTask(lastTask.id).then((_) {
                          _fetchTasks();
                        });
                      }
                    },
                    child: Text("Delete Task"),
                  ),
                ),
              ],
            ),

            Expanded(
              child: ListView.builder(
                itemCount: _tasks.length,
                itemBuilder: (context, index) {
                  final task = _tasks[index];
                  return Card(
                    color: task.isCompleted ? Colors.green[100] :
Colors.white,
                    elevation: 2.0,
                    child: ListTile(
                      title: Text(task.title),
                      subtitle: Text(task.description),
                      trailing: IconButton(
                        icon: Icon(Icons.edit),
                        onPressed: () {
                        },
                      ),
                      onTap: () => {
                        titleController.text = task.title,
                        descriptionController.text = task.description,
                      },
                    ),
                  );
                },
```

```
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

❖ Bước 5: xây dựng hàm main.dart

```dart
import 'package:flutter/material.dart';
import 'package:lab9_flutter_sqlite/views/task_screen.dart';

void main() {
  runApp(const MyApp());
}

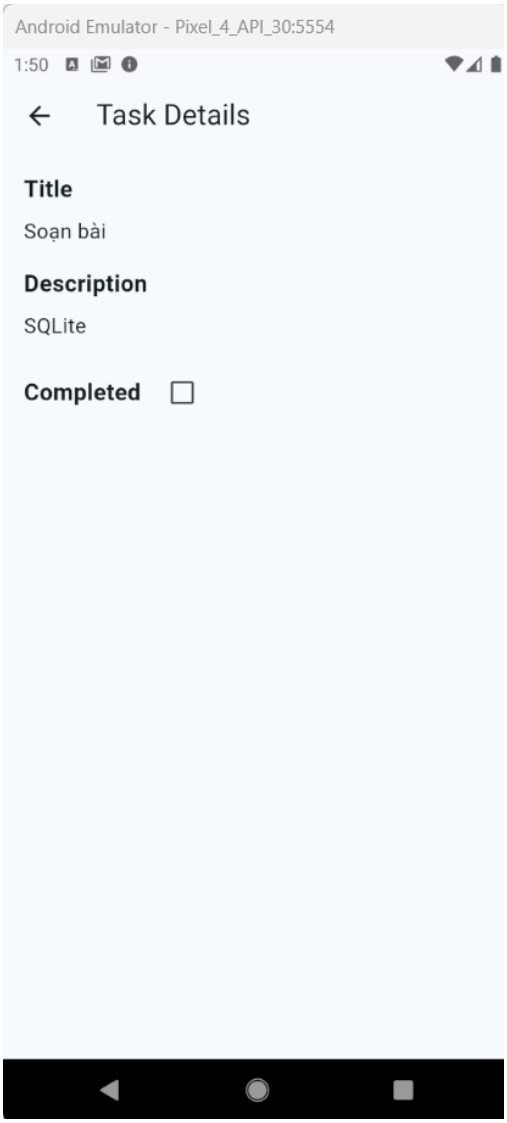
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const
Color.fromARGB(255, 135, 200, 230)),
      ),
      debugShowCheckedModeBanner: false,
      home:TaskScreen(),
    );
  }
}
```

3. Bài tập thực hành tại lớp

Bài tập 1. Hãy xây dựng màn hình cập nhật trạng thái công việc có giao diện như hình bên dưới, với mục đích lấy thông tin của công việc được chọn từ màn hình TaskScreen.

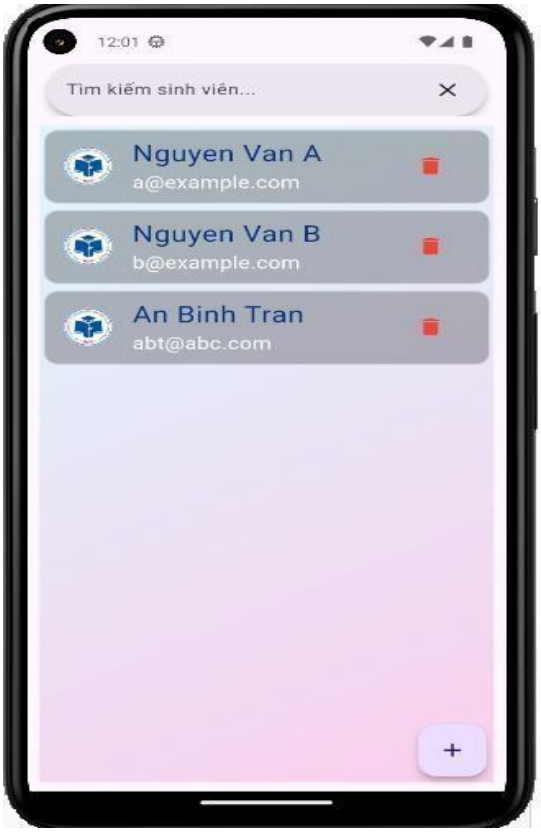
Khi nhấp vào checkbox Completed để cập nhật trạng thái của công việc và quay trở về màn hình TaskScreen.



Bài tập 2. Hãy xây dựng ứng dụng quản lý sinh viên, biết rằng thông tin sinh viên được mô tả như sau:

```
class SinhVien {
     final int? id;
     final String name;
     String email;
     SinhVien({this.id, required this.name, required
this.email});
     Map<String, dynamic> toMap() {
          return {'id': id, 'name': name, 'email': email};
     }
     factory SinhVien.fromMap(Map<String, dynamic> map) {
          return SinhVien(id: map['id'], name: map['name'],
          email: map['email']);
          }
}
```

Hãy xây dựng màn hình quản lý sinh viên như sau:





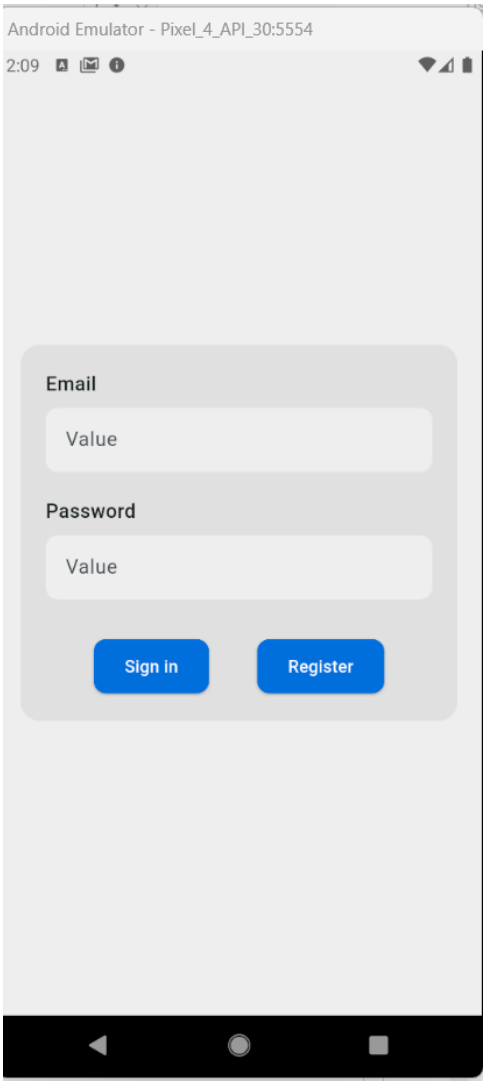


Khi nhấn vào button '+' sẽ hiển thị màn hình cho phép nhập thông tin của một sinh viên mới.; nhấp "Lưu" sẽ lưu vào cơ sở dữ liệu và trở về màn hình chính hiển thị thêm thông tin của sinh viên mới thêm vào, nhấp "Hủy" sẽ quay về màn hình chính của ứng dụng. Khi nhấn vào thông tin của sinh viên trên màn hình sẽ hiển thị một màn hình thông tin chi tiết của sinh viên cho phép cập nhật thông tin của sinh.

C. BÀI TẬP VỀ NHÀ

Bài tập 1. Hãy xây dựng chương trình quản lý chi tiêu cá nhân, mỗi mục chi tiêu có các thông tin: nội dung chi tiêu, số tiền, ghi chú.

Bài tập 2. Hãy xây dựng form đăng nhập và đăng ký có giao diện cơ bản như sau:



- Nếu đã có tài khoản, chọn "sign in" để đăng nhập, hệ thống sẽ kiểm tra tài khoản người dùng (email, password), nếu thông tin trùng khớp sẽ thông báo đăng nhập thành công.
- Nếu chưa có tài khoản, chọn "Register" để đăng ký tài khoản.